

Joint Detection and Decoding: A Graph Neural Network Approach

Jannis Clausius *Graduate Student Member, IEEE*, Marvin Rübenacke *Graduate Student Member, IEEE*, Daniel Tandler *Graduate Student Member, IEEE*, Stephan ten Brink *Fellow, IEEE*

Abstract—Narrowing the performance gap between optimal and feasible detection in inter-symbol interference (ISI) channels, this paper proposes to use graph neural networks (GNNs) for detection that can also be used to perform joint detection and decoding (JDD). For detection, the GNN is build upon the factor graph representations of the channel, while for JDD, the factor graph is expanded by the Tanner graph of the parity-check matrix (PCM) of the channel code, sharing the variable nodes (VNs). A particularly advantageous property of the GNN is a) the robustness against cycles in the factor graphs which is the main problem for sum-product algorithm (SPA)-based detection, and b) the robustness against channel state information (CSI) uncertainty at the receiver. Consequently, a fully deep learning-based receiver enables joint optimization instead of individual optimization of the components, so-called end-to-end learning. Furthermore, we propose a parallel flooding schedule that also reduces the latency, which turns out to improve the error correcting performance. The proposed approach is analyzed and compared to state-of-the-art baselines for different modulations and codes in terms of error correcting capability and latency. The gain compared to SPA-based detection might be explained with improved messages between nodes and adaptive damping of messages. For a higher order modulation in a high-rate turbo detection and decoding (TDD) scenario the GNN shows a, at first glance, surprisingly high gain of 6.25 dB compared to the best, feasible non-neural baseline.

Index Terms—Graph neural networks, detection, turbo detection, equalization, deep learning.

I. INTRODUCTION

INTER-SYMBOL interference (ISI) can appear in many scenarios, e.g., band-limited channels after filtering at the receiver [2] or single carrier transmission systems used in low power communication [3]. In recent years, neural network (NN)-based algorithms entered the landscape of detection for inter-symbol interference (ISI) channels besides the maturity of a posteriori probability (APP)-based and linear minimum mean square error (LMMSE)-based algorithms. APP-based approaches usually yield good performance, but suffer from high complexity. In contrast, LMMSE-based algorithms usually are of low complexity, but struggle with achieving the optimal maximum a posteriori (MAP) performance. The neural contender may offer a solution in between with good performance, yet feasible complexity. While this class of detection algorithms have been introduced already 35 years ago [4], the

increase in computing power has renewed interest in it. Model-based NNs appear to be particularly attractive, as they exploit domain knowledge [5] and thereby promise lower complexity, better generalization, and require smaller training data sets than their purely data-driven counterparts.

In [5], they are categorized into *model-aided NNs* and *NN-aided inference*. Model-aided NNs consist of handcrafted NNs for a given problem. They may share similarities with classical algorithms, e.g., convolutional neural network (CNN)-based detection [6], [7] is similar to the LMMSE algorithm and recurrent neural network (RNN)-based detection [8]–[10] resembles the Bahl–Cocke–Jelinek–Raviv (BCJR) [11] algorithm. In contrast, in NN-aided inference, a classical algorithm is augmented by learned components, e.g., using multilayer perceptrons (MLPs) as factor nodes (FNs) in the BCJR algorithm [12]. A particularly well suited basis for NN-aided inference are factor graphs based on the Forney factor graph (FFG) [13] or the Ungerboeck factor graph (UFG) [14]. Here, the sum-product algorithm (SPA) yields a sub-optimal performance [15]–[17], opening up the possibility of neural augmentation. For instance, in [18], a neural node was added to the factor graph, and in [19], [20], trainable weights were added. However, these neural augmentations might not suffice in cases of strong oscillation and amplification caused by severe ISI.

By proposing to replace nodes and edges by MLPs, we continue this path resulting in a graph neural network (GNN) [21] right on the border between model-based NNs and NN-aided inference. GNNs were successfully applied in channel decoding [22]–[24] and multiple-input multiple-output (MIMO) detection [25]–[27]. We believe GNNs are a great fit for detection in ISI channels. Firstly, they promise robust computations to mitigate the negative effect of the vast amount of short cycles in the factor graphs for SPA. In particular, increasing robustness to amplification and oscillation. Secondly, in contrast to the MIMO scenario, the sparse connectivity between nodes in the FFG and UFG reduces the complexity of the GNN drastically, making it computationally attractive. For MIMO, the GNNs use fully connected adjacency matrices [25], [27]. This work aligns with GNNs based on factor graphs, as in [22], [24], but also considers the detection problem as part of the factor graph, enabling joint detection and decoding (JDD) and various schedules for message passing. Note that GNNs are not unique in leveraging the local structure of the ISI problem for efficient detection. RNNs and CNNs can do that as well [6], [10].

However, in contrast to RNNs or CNNs, GNNs can also leverage the sparse nature of a low-density parity-check (LDPC) code for decoding [24] enabling JDD and improving

J. Clausius, M. Rübenacke, D. Tandler and S. ten Brink are with the Institute of Telecommunications, University of Stuttgart, 70569 Stuttgart, Germany (e-mail: clausius@inue.uni-stuttgart.de; ruebenacke@inue.uni-stuttgart.de; tandler@inue.uni-stuttgart.de; tenbrink@inue.uni-stuttgart.de) Parts of this paper have been presented at 2024 IEEE Int. Symp. Inf. Theory (ISIT) [1]. This work is supported by the German Federal Ministry of Education and Research (BMBF) within the project Open6GHub (grant no. 16KISK019).

TABLE I: List of acronyms

BCE	binary cross-entropy	BMI	bit-wise mutual information
CIR	channel impulse response	CCT	constant channel transformation
CSI	channel state information	CNN	convolutional neural network
FN	factor node	FFG	Forney factor graph
FGNN	factor graph neural network	GNN	graph neural network
ISI	inter-symbol interference	JDD	joint detection and decoding
MLP	multilayer perceptron	NN	neural network
RNN	recurrent neural network	SDD	separate detection and decoding
SPA	sum-product algorithm	TDD	turbo detection and decoding
UFG	Ungerboeck factor graph	VN	variable node

receiver performance over separate detection and decoding (SDD). JDD discriminates from SDD by code/decoder aware detection or detection aware decoding. One practical way to approach the asymptotic performance limit of JDD is using SDD components and allowing feedback from the decoder to the detector, resulting in turbo detection and decoding (TDD) [28]. However, in the non-asymptotic regime (i.e., for finite block lengths), JDD can be superior to TDD by leveraging joint optimization instead of separate optimization of the components. The concepts are visualized in Fig. 1. JDD based on a model-free approach was investigated in [29], however, limited to ultra-short block lengths ($N = 16$). In contrast, the model-based approach in [30] scales to longer block lengths, but is limited by the exponential nature of the BCJR algorithm.

For short block lengths ($N = 128$), we propose a true JDD scheme that combines two GNNs; one for detection and one for decoding. Both GNNs are connected by their variable nodes (VNs) to form a combined factor graph that allows information exchange between detection and decoding in both directions at the same time, called the flooding schedule. The joint GNN constitutes a fully deep learning-based receiver that can be jointly optimized in an end-to-end fashion resulting in a JDD scheme [7], [29]. Since the connectivity in the graphs for detection and decoding are sparse, respectively, the joint graph is also sparsely connected. This enables the efficient use of GNNs. In [31], such a combined graph was used with a decision feedback detector and an SPA decoder. For longer block lengths, we resort to a TDD scheme where only detection is performed by a GNN and decoding is performed by the conventional SPA. This means, the GNN is trained to output log-likelihood ratios (LLRs) and is neither aware of the code nor fine-tuned to the decoder.

Note that throughout the paper, we demonstrate NNs that are not optimized in terms of complexity. The purpose is to show the performance limit that a neural architecture can achieve. Thus, the NNs are over-parameterized enabling smooth training and reproducible results. Our contributions are as follows:

- We demonstrate that GNNs are capable of matching the MAP detection performance for severe ISI with binary phase shift keying (BPSK) and outperforming state-of-the-art baselines with higher order modulation. Further, the robustness of the GNN towards channel state information (CSI) uncertainty at the receiver is demonstrated, i.e., noisy channel impulse response (CIR), and towards

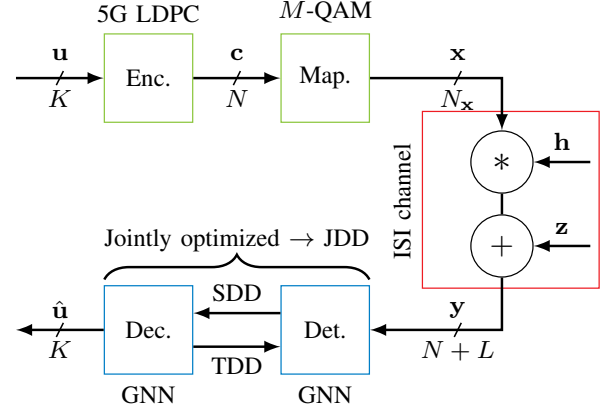


Fig. 1: Block diagram of the system model with an ISI channel (convolution with channel impulse response (CIR) $\mathbf{h}$ and addition of noise $\mathbf{z}$) and graph neural networks (GNNs) for separate detection and decoding (SDD), turbo detection and decoding (TDD) and JDD.

changing distributions of the CIR.

- We analyze the learned messages in the GNN and give intuitive explanations for the performance gains.
- We propose GNNs for JDD where the respective factor graphs are connected by their common VNs and allow joint processing. The proposed approach is compared to state-of-the-art baselines.
- Finally, we propose a flooding schedule for the GNNs on the joint factor graph instead of an iterative/sequential schedule, improving both performance and latency.

II. PRELIMINARIES

TABLE II: List of variables

$\mathcal{F}$	set of FNs	$\mathbf{c}$	coded bits
$\mathbf{g}$	attributes	$\mathbf{h}$	channel impulse response
$\mathbf{H}$	channel matrix	I	mutual information
K	number of uncoded bits	$\mathcal{L}$	loss function
L	channel memory	ℓ	log-likelihood ratios
$\mathbf{m}$	messages	M	modulation alphabet size
N_x	number of symbols	N	number of coded bits
N_{it}	number of iterations	R	achievable rate
R_C	code rate	$\mathbf{s}$	states
$\boldsymbol{\theta}$	trainable variables	$\hat{\mathbf{u}}$	estimated uncoded bits
$\mathbf{u}$	uncoded bits	$\mathcal{V}$	set of VNs
$\mathbf{x}$	symbols	$\mathbf{y}$	channel observation

A. System Overview

Fig. 1 shows the block diagram of the considered system model. We assume 5G LDPC channel encoding of the bits $\mathbf{u} \in \{0, 1\}^K$ with code dimension K into codeword $\mathbf{c} \in \{0, 1\}^N$ bits of length N and M -quadrature amplitude modulation (QAM) into the transmission symbols $\mathbf{x} \in \{0, 1\}^{N_x}$ of length N_x , where M denotes the alphabet size. Note that $M = 2$ and $M = 4$ results in BPSK and quadrature phase shift keying (QPSK), respectively. The ISI channel is modeled as a tapped delay line (TDL) with CIR $\mathbf{h}$ and additive white Gaussian noise (AWGN). Hence, the discrete-time channel can be mathematically described by [13]

$$y_i = \sum_{l=0}^L h_l x_{i-l} + z_i \quad (1)$$

with channel memory L , transmitted coded symbols x_i and where $z_l \sim \mathcal{CN}(0, \sigma^2)$ denotes independent and identically distributed (i.i.d.) Gaussian noise. Furthermore, we assume that the symbols x_i for $i \in [-L+1, -1] \cup [N_x-1, N_x+L]$ are known at the receiver, i.e., fixed to zero, where N_x denotes the transmission length. Thus, we can rewrite equation (1) to $\mathbf{y} = \mathbf{H}\tilde{\mathbf{x}} + \mathbf{z}$ with the Toeplitz matrix $\mathbf{H} \in \mathbb{C}^{(N_x+L) \times (N_x+2L)}$ called the *channel matrix*, equivalent transmit sequence $\tilde{\mathbf{x}} \in \mathbb{C}^{N_x+2L}$ and $\mathbf{z} \in \mathbb{C}^{N_x+L}$. The goal of the receiver is to estimate the bit-wise APPs $P(u_i|\mathbf{y}, \mathbf{H})$ in the form of log-likelihood ratios (LLRs) $\ell_i = \log \frac{P(u_i=0|\mathbf{y}, \mathbf{H})}{P(u_i=1|\mathbf{y}, \mathbf{H})}$.

B. Sum-Product Algorithm

Consider the factorization of a joint probability function $f(\mathcal{X})$ into a product of J factors $g_j(\mathcal{X}_j)$, i.e., $f(\mathcal{X}) = \prod_{1 \leq j \leq J} g_j(\mathcal{X}_j)$, where $\mathcal{X}$ corresponds to a set of random variables $\{x_1, \dots, x_N\}$ and $\mathcal{X}_j \subset \mathcal{X}$. One can construct a corresponding factor graph consisting of VNs $\mathcal{V}$ and factor nodes $\mathcal{F}$ by assigning each variable x_j a VN $V_j \in \mathcal{V}$, each function g_i a FN $F_i \in \mathcal{F}$, and introducing an edge connecting between V_j and F_i if x_j is an argument of g_i [15]. The marginals of f , i.e., $f(x_i)$ for each $x_i \in \mathcal{X}$, can be efficiently computed using the message passing algorithm SPA on the factor graph. The algorithm operates by sending messages from VNs/FNs to FNs/VNs, denoted as $m_{V_i \rightarrow F_j} / m_{F_j \rightarrow V_i}$. In the log-domain, the messages are determined using

$$m_{V_i \rightarrow F_j} = \sum_{F_k \in \mathcal{F}(V_i) \setminus \{F_j\}} m_{F_k \rightarrow V_i}$$

$$m_{F_j \rightarrow V_i} = \max_{\sim \{V_i\}}^* \left(\ln(g_j(\mathcal{X}_j)) + \sum_{V_k \in \mathcal{V}(F_j) \setminus \{V_i\}} m_{V_k \rightarrow F_j} \right)$$

where $\mathcal{V}(F_j)$ and $\mathcal{F}(V_i)$ denote the *neighborhood* of VNs of F_j and of FNs of V_i , respectively and $\max_{\sim \{V_i\}}^*$ denotes the Jacobian logarithm [32] applied over all messages except $m_{V_i \rightarrow F_j}$. The Jacobian logarithm is defined as $\max^*(a_1, \dots, a_N) = \log(e^{a_1} + \dots + e^{a_N})$.

C. Factor Graph-based Detection

With an appropriate factorization of the joint symbol APP $P(\mathbf{x}|\mathbf{y}, \mathbf{H})$, one can obtain an equivalent factor graph representation. Here, the FNs correspond to $\mathbf{H}\tilde{\mathbf{x}}$ and the VNs to the symbols in $\tilde{\mathbf{x}}$ (the first and last L VNs are *virtual* VNs). SPA is used to calculate the symbol-wise APP $P(x_i|\mathbf{y}, \mathbf{H})$. Using Bayes' theorem, we obtain

$$P(\mathbf{x}|\mathbf{y}) \propto P(\mathbf{y}|\mathbf{x})P(\mathbf{x}) \propto \exp\left(-\frac{\|\mathbf{y} - \mathbf{H}\tilde{\mathbf{x}}\|^2}{2\sigma^2}\right). \quad (2)$$

Direct application of the chain rule yields

$$P(\mathbf{y}|\mathbf{x}) \propto \prod_{i=1}^{N_x} \exp\left(-\frac{\|y_i - \sum_{l=0}^L h_l \tilde{x}_{i-l}\|^2}{2\sigma^2}\right). \quad (3)$$

The factor graph corresponding to the factorization of equation (3) was first proposed in [13] and is referred to as FFG in the following. An example for the FFG is shown in Fig. 3. Intuitively, each FN corresponds to a channel observation and

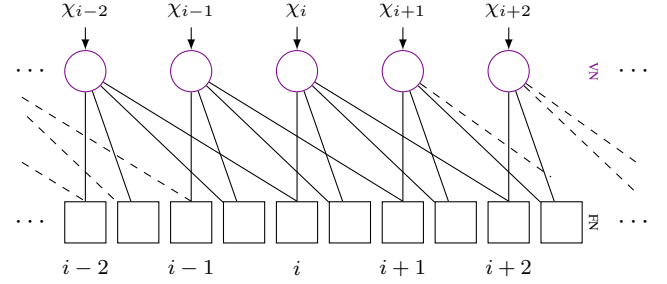


Fig. 2: Ungerboeck factor graph (UFG) for a channel with memory $L = 2$. Input to the VNs are the estimates $\chi := \mathbf{H}^H \mathbf{y}$. The graph is sparse, since the FN degree is $d_F = 2$ and the VN degree is $d_V = 2L$.

is connected to the $L+1$ VNs that directly impact the channel observation. In other words, the rows of $\mathbf{H}$ correspond to VNs, the columns to FNs and are connected if the corresponding entry in $\mathbf{H}$ is non-zero.

By rewriting equation (2) and using the substitutions $\mathbf{G} := \mathbf{H}^H \mathbf{H}$ and $\chi := \mathbf{H}^H \mathbf{y}$ we obtain an alternative factorization of the APP [14], [17]:

$$P(\mathbf{y}|\mathbf{x}) \propto \prod_{i=1}^{N_x} \left[F_i^{\text{UFG}}(x_i) \prod_{j=1, j \neq i}^N I_{i,j}(x_i, x_j) \right]$$

$$F_i^{\text{UFG}}(x_i) := \exp\left(\frac{1}{2\sigma^2}(2\chi_i x_i - G_{i,i}|x_i|^2)\right) \quad (4)$$

$$I_{i,j}(x_i, x_j) := \exp\left(-\frac{1}{2\sigma^2}G_{i,j}x_i x_j\right).$$

The corresponding factor graph is referred to as UFG in the following and is shown in Fig. 2. In the UFG, two VNs are connected via a FN, if their corresponding symbols interfere with each other, i.e. they are no more than $L+1$ positions apart. The distinctive feature compared to the FFG is that a FN only connects two VNs instead of $L+1$.

Applying SPA to these factor graphs is not guaranteed to converge, since ISI channels may contain short cycles in their graphs [16]. However, for ISI channels with $L \ll N_x$ the graphs are sparse allowing for efficient graph-based processing. The number of edges divided by the maximal possible number of edges scales with $\frac{L}{N_x}$, disregarding constants.

D. EXIT Charts

To visualize and analyze the behavior of iterative systems, i.e., TDD, we use Extrinsic information transfer (EXIT) charts [33]. In this case, we iterate between two components: a detector and a decoder. The information gain is quantified by the extrinsic mutual information $I_E = I(\mathbf{c}; \ell_E)$ of the extrinsic LLRs ℓ_E and the transmitted codeword $\mathbf{c}$ that is tracked over multiple iterations. As long as for both components the information gain is larger than the *a priori* information I_A , the iterative process increases information. Assuming that we know the transfer characteristics $T(\cdot)$ of the detector $I_E = T_{\text{Det}}(I_A, \gamma)$, where γ is the signal-to-noise-ratio (SNR), and the decoder $I_E = T_{\text{Dec}}(I_A)$, the EXIT chart plots I_E over I_A for the detector and I_A over I_E for the decoder, e.g., as in Fig. 12. Decoding is successful if $T_{\text{Dec}}(I_A) \rightarrow 1$ before T_{Det} and T_{Det} intersect.

In general, the transfer characteristic $T(I_A, \cdot)$ of a component can be derived analytically or empirically. For most detectors and decoders, an analytical description of $T(\cdot)$ is unknown, thus, we resort to an empirical one. To empirically evaluate $I_E = T(I_A, \cdot)$, we need to compute the extrinsic LLRs

$$[\ell_E]_i = [f(\ell_{A, \sim i}, \cdot)]_i$$

where $[\cdot]_i$ denotes the element with index i , $f(\cdot)$ is the processing of the component, e.g., a detector or a decoder, $\ell_{A, \sim i}$ is the vector of *a priori* LLRs with average mutual information I_A omitting index i . In other words, the component $f(\cdot)$ needs to be evaluated once for each position i with $\ell_i = 0$. For true APP components, the calculation of ℓ_E can be simplified to

$$\ell_E = \ell_T - \ell_A = f(\ell_A, \cdot) - \ell_A, \quad (5)$$

where ℓ_T refers to the total LLRs.

During inference of a considered system, ℓ_A are the extrinsic LLRs from the respective other component. However, we would like to characterize the component in question $T(I_A, \cdot)$ without any influence of the other component. To circumvent this, the *a priori* LLRs ℓ_A can be modeled as a BPSK modulated side channel transmission, where the SNR is chosen according to the desired *a priori* information I_A . The resulting LLRs can be sampled from

$$\ell_A \sim \mathcal{N}((1 - 2\mathbf{u}) \cdot \mu(I_A), 2\mu(I_A)\mathbf{I}) \quad (6)$$

where, $\mathbf{u} \in \{0, 1\}^K$ are the transmitted bits and $\mu(I_A) \approx \frac{1}{2} \left(-\frac{1}{H_1} \log_2 \left(1 - I_A^{\frac{1}{H_3}} \right) \right)^{\frac{1}{H_2}}$ with constants $H_1 = 0.3073$, $H_2 = 0.8935$ and $H_3 = 1.1064$ [34]. Finally, the extrinsic bit-wise mutual information (BMI) can be estimated by [35]

$$I_E = 1 - \mathbb{E}_{\mathbf{u}, \ell_E} \left\{ \log_2(1 + e^{-(1-2\mathbf{u}) \cdot \ell_E}) \right\}.$$

E. Information Rates

The achievable transmission rate of JDD is given by the average mutual information [36]

$$R_{\text{JDD}} = \frac{1}{K} I(\mathbf{U}; \hat{\mathbf{U}}) = \frac{1}{K} (H(\mathbf{U}) - H(\mathbf{U}|\hat{\mathbf{U}})),$$

where $H(\mathbf{U})$ is the joint entropy of the random variable $\mathbf{U}$ associated with the uncoded bits and $H(\mathbf{U}|\hat{\mathbf{U}})$ is the conditional entropy of $\mathbf{U}$ given the estimate $\hat{\mathbf{U}}$. Due to the complexity reduction of the divide and conquer approach, SDD is commonly used in practice, which relates in the limit

$$\begin{aligned} R_{\text{JDD}}^* &= \lim_{K \rightarrow \infty} \frac{1}{K} (H(\mathbf{U}) - H(\mathbf{U}|\hat{\mathbf{U}})) \\ &\geq \lim_{K \rightarrow \infty} \frac{1}{K} \sum_i (H(U_i) - H(U_i|\hat{\mathbf{U}})) \\ &= R_{\text{SDD}}^*. \end{aligned}$$

If all U_i and U_j are conditionally independent knowing $\hat{\mathbf{U}}$, meaning $H(U_i|\hat{\mathbf{U}}, U_j) = H(U_i|\hat{\mathbf{U}}) \forall i \neq j$, then equality follows.

TDD is a scheme to approach R_{JDD}^* with methods from SDD. The area theorem [37] states that the area under

the curve of the EXIT characteristic of an optimal detector $T_{\text{Det}}^*(I_A)$ is equal to the JDD rate

$$R_{\text{JDD}}^* \geq R_{\text{TDD}}^* = \int_0^1 T_{\text{Det}}^*(I_A) dI_A.$$

While equality only holds for an optimal detector and if I_A comes from an binary erasure side channel, practice showed that AWGN side channel information, which resembles soft feedback from a decoder, results in approximately the same characteristic. Furthermore, to achieve R_{JDD}^* , an optimal channel code of an infinite length with a matched EXIT characteristic $T_{\text{Dec}}^*(I_A)$ is required. Here, matched means that $T_{\text{Det}}^*(I_A) - T_{\text{Dec}}^{-1}(I_A) \rightarrow 0^+$. Since $T_{\text{Det}}^*(I_A)$ is weakly monotone increasing in I_A

$$\begin{aligned} R_{\text{JDD}}^* &\geq R_{\text{TDD}}^* = \int_0^1 T_{\text{Det}}^*(I_A) dI_A \\ &\geq \int_0^1 T_{\text{Det}}^*(0) dI_A = T_{\text{Det}}^*(0) = R_{\text{SDD}}^*. \end{aligned}$$

Note that in the following, we take the point of view of a detector, meaning we assume i.i.d. symbols X_i resulting in $H(\mathbf{X}) = \sum_i H(X_i)$. Until here we assumed optimal components, e.g., MAP detectors with output distribution $P_i = P_{\text{MAP}}(X_i|\mathbf{Y}, I_A)$. However, in this work we explore non-optimal detectors with output distribution $Q_i = P_{\text{Det}}(X_i|\mathbf{Y}, I_A)$. The achievable rate [38], [39] is then

$$\begin{aligned} R &= \frac{1}{N_{\mathbf{x}}} \sum_i I(X_i; Q_i) = \frac{1}{N_{\mathbf{x}}} \sum_i (H(X_i) - \mathcal{L}(X_i, Q_i)) \\ &= \frac{1}{N_{\mathbf{x}}} \sum_i (I(X_i; \mathbf{Y}|I_A) - D_{\text{KL}}(P_i||Q_i)) \end{aligned}$$

where $\mathcal{L}(X_i, Q_i)$ is the cross-entropy function and $D_{\text{KL}}(\cdot||\cdot)$ is the Kullback-Leibler divergence. For $I_A = 0$, it follows $R_{\text{SDD}}^* - R = \sum_i D_{\text{KL}}(P_i||Q_i)$ and for turbo detection $R_{\text{TDD}}^* - R = \sum_i \int_0^1 D_{\text{KL}}(P_i||Q_i) dI_A$, quantifying the rate loss in terms of the difference between the optimal output distribution P_i and the output distribution of the detector Q_i .

Finally, as in practice bit-metric decoding is commonly used, e.g., in mobile communication standards, we are interested in the BMI regarding the bits C . This means the measure of interest for the bit estimates Q_i^{Bit} is

$$\begin{aligned} R_{\text{SDD}}^{\text{BMI}} &= \frac{1}{N} \sum_i I(C_i|\mathbf{Y}) \\ &= 1 - \frac{1}{N} \sum_i \mathcal{L}_{\text{BCE}}(C_i, Q_i^{\text{Bit}}) \end{aligned} \quad (7)$$

$$\begin{aligned} R_{\text{TDD}}^{\text{BMI}} &= \int_0^1 T(I_A) dI_A \\ &= \int_0^1 1 - \frac{1}{N} \sum_i \mathcal{L}_{\text{BCE}}(C_i, Q_i^{\text{Bit}}) dI_A, \end{aligned} \quad (8)$$

where $\mathcal{L}_{\text{BCE}}$ is the binary cross-entropy (BCE).

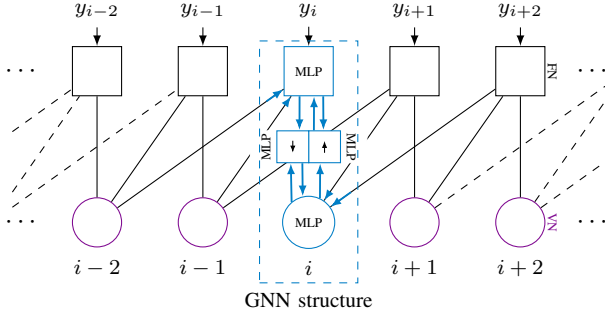


Fig. 3: Forney factor graph (FFG) with GNN elements are presented for an exemplary pair of variable node (VN)/factor node (FN) for a channel with memory $L = 2$. Blue nodes represent multilayer perceptrons (MLPs). For LDPC decoding the FNs are usually referred to as check nodes and the channel observations $\mathbf{y}$ are input to the VNs.

III. GRAPH NEURAL NETWORKS FOR DETECTION

In this section, we describe the application of GNNs for detection of ISI channels based on the FFG and the UFG. This resembles a SDD scenario, where we consider the detection GNNs independently of the channel code assuming i.i.d. inputs bits and no a priori information ($I_A = 0$) from the decoder. Thus, we evaluate the achievable rate, shown in equation (7), as measure of soft output quality of the detection, giving a good indicator on the performance, if one would use an outer channel code.

A. Graph Neural Network-based Detection

The bipartite GNN framework from [24] can be directly applied to the FFG and UFG for detection. In each iteration of the GNN, first the FNs are updated, then the edges towards VNs, followed by the VNs themselves, and finally the edges towards FNs. As shown in Fig. 3, the GNN assigns learnable parametric functions $f_{\theta}(\cdot)$ to each VN, each FN and each directed edge. Here, this parametric function is implemented by a MLP with trainable weights θ . The weights θ are shared for each type of node or directed edge in the graph, i.e., there is a set θ_V for VNs, θ_F for FNs, $\theta_{F \rightarrow V}$ for edges directed towards VNs, and $\theta_{V \rightarrow F}$ for edges directed towards FNs. Each MLP represents a node state update function or an edge state update function. The state of a node or edge is represented by a d -dimensional vector, where d is the so called *feature size*. For simplicity, all nodes and edges share the same value of d . The inference is summarized in Algorithm 1. The updated state $\mathbf{s}_{V_i}^{(t+1)}$ in iteration $t + 1$ of VN V_i is calculated by

$$\mathbf{s}_{V_i}^{(t+1)} = f_{\theta_V} \left(\mathbf{s}_{V_i}^{(t)}, \frac{1}{|\mathcal{F}(V_i)|} \sum_{F_j \in \mathcal{F}(V_i)} \mathbf{m}_{F_j \rightarrow V_i}, \mathbf{g}_{V_i} \right), \quad (9)$$

where $\mathcal{F}(V_i)$ is the set of connected FNs, $\mathbf{m}_{F_j \rightarrow V_i}$ is the state of the edge from F_j to V_i , and $\mathbf{g}_{V_i} \in \mathbb{R}^d$ is a trainable attribute of node V_i . In a similar fashion, the update of FN states $\mathbf{s}_{F_j}^{(t+1)}$ is calculated by

$$\mathbf{s}_{F_j}^{(t+1)} = f_{\theta_F} \left(\mathbf{s}_{F_j}^{(t)}, \frac{1}{|\mathcal{V}(F_j)|} \sum_{V_i \in \mathcal{V}(F_j)} \mathbf{m}_{V_i \rightarrow F_j}, \mathbf{g}_{F_j} \right), \quad (10)$$

Algorithm 1: GNN-Inference

Input : Channel observations $\mathbf{y}$
Trainable parameters θ :
- Input embedding $\mathbf{W}$
- Read-out $\mathbf{v}$
- Variable to factor weights $\theta_{V \rightarrow F}$
- Factor to variable weights $\theta_{F \rightarrow V}$
- Variable attributes $\{\mathbf{g}_{V_i}\} \forall V_i \in \mathcal{V}$
- Factor attributes $\{\mathbf{g}_{F_i}\} \forall F_i \in \mathcal{F}$
- Message attributes $\{\mathbf{g}_{V_i \rightarrow F_j}, \mathbf{g}_{F_j \rightarrow V_i}\} \forall (V_i, F_j) \in (\mathcal{V}, \mathcal{F}(V_i))$

Output: ℓ // LLRs

```

// Embedding
 $\mathbf{s}_{V_i}^{(1)} \leftarrow \mathbf{0}, \quad \forall V_i \in \mathcal{V}$ 
 $\mathbf{s}_{F_i}^{(1)} \leftarrow \mathbf{W} \cdot [\text{real}(y_i), \text{imag}(y_i)]^T, \quad \forall F_i \in \mathcal{F}$ 
// Message passing
For  $t \in [1, \dots, N_{\text{It}}]$  do
  // Factor to variable messages
  For  $(V_i, F_j) \in (\mathcal{V}, \mathcal{F}(V_i))$  do
     $\mathbf{m}_{F_j \rightarrow V_i} \leftarrow f_{\theta_{F \rightarrow V}}(\mathbf{s}_{F_j}^{(t)}, \mathbf{s}_{V_i}^{(t)}, \mathbf{g}_{F_j \rightarrow V_i})$ 
  end
  // Variable nodes
  For  $V_i \in \mathcal{V}$  do
     $\mathbf{s}_{V_i}^{(t+1)} \leftarrow f_{\theta_V}(\mathbf{s}_{V_i}^{(t)}, \frac{1}{|\mathcal{F}(V_i)|} \sum_{F_j \in \mathcal{F}(V_i)} \mathbf{m}_{F_j \rightarrow V_i}, \mathbf{g}_{V_i})$ 
  end
  // Variable to factor messages
  For  $(V_i, F_j) \in (\mathcal{V}, \mathcal{F}(V_i))$  do
     $\mathbf{m}_{V_i \rightarrow F_j} \leftarrow f_{\theta_{V \rightarrow F}}(\mathbf{s}_{V_i}^{(t+1)}, \mathbf{s}_{F_j}^{(t)}, \mathbf{g}_{V_i \rightarrow F_j})$ 
  end
  // Factor nodes
  For  $F_j \in \mathcal{F}$  do
     $\mathbf{s}_{F_j}^{(t+1)} \leftarrow f_{\theta_F}(\mathbf{s}_{F_j}^{(t)}, \frac{1}{|\mathcal{V}(F_j)|} \sum_{V_i \in \mathcal{V}(F_j)} \mathbf{m}_{V_i \rightarrow F_j}, \mathbf{g}_{F_j})$ 
  end
end
// Read-out
For  $V_i \in \mathcal{V}$  do
   $\ell_i \leftarrow \mathbf{v}^T \mathbf{s}_{V_i}$ 
end

```

where $\mathcal{V}(F_j)$ is the set of connected VNs to FN F_j . Between the VN and FN updates, the edges are updated according to

$$\begin{aligned} \mathbf{m}_{V_i \rightarrow F_j} &= f_{\theta_{V \rightarrow F}}(\mathbf{s}_{V_i}, \mathbf{s}_{F_j}, \mathbf{g}_{V_i \rightarrow F_j}), \\ \mathbf{m}_{F_j \rightarrow V_i} &= f_{\theta_{F \rightarrow V}}(\mathbf{s}_{F_j}, \mathbf{s}_{V_i}, \mathbf{g}_{F_j \rightarrow V_i}). \end{aligned}$$

While the node states $\mathbf{s}$ and message states $\mathbf{m}$ are updated during inference, the attributes $\mathbf{g}$ are constant during inference, but are learned during training. For initialization a normal distribution is chosen $\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. In [24], the node and edge attributes were set to $\mathbf{0}$ as they did not improve the

performance for decoding linear channel codes. However, in the case of detection, the edge attributes are crucial, since different edges fulfill different tasks, i.e., in classical SPA detection the messages originating from a FN are calculated with different functions. Each edge function represents a different tap of the CIR, thus, needs a marginalization over different variables. The taps are the same for all FNs, meaning that not every edge needs an individual attribute vector. The periodicity of the edge type is $N_p = \max(d_F, d_V)$ where d_F and d_V are the factor node degree and variable node degree, respectively. As a result, the number of individual feature vectors is $N_p = L + 1$ for the FFG and $N_p = 2L$ for the UFG. This enables length flexibility in N_x of the GNN, since nodes and edges can be added to the graph with the same weights as the existing ones.

Before and after inference on the graph, the channel outputs are projected to the feature space of the GNN and the inference results are projected to LLRs, respectively. Before inference, $\mathbf{y}$ is linearly projected to the d dimensional FN state $\mathbf{s}_{F_i} = \mathbf{W} \cdot [\text{real}(y_i), \text{imag}(y_i)]^T$, where $\mathbf{W} \in \mathbb{R}^{d \times 2}$ is a trainable projection matrix. The VN states are initialized with $\mathbf{s}_{v_i} = \mathbf{0}$. After inference, the VN states are projected to LLRs by $\ell_i = \mathbf{v}^T \mathbf{s}_i$, where $\mathbf{v} \in \mathbb{R}^d$ denotes a learnable projection vector. All MLPs and trainable matrices are initialized by a normal Glorot distribution [40]. Finally, a sigmoid function $\sigma_{\text{sigmoid}}(\cdot)$ converts the LLRs to probabilities $q_i = \sigma_{\text{sigmoid}}(\ell_i)$. Note that the conversion from feature vectors to probabilities can be done after any number of iterations, allowing for adjustment to the desired latency, complexity, or performance.

Further, instead of using a normalized sum for the aggregation of messages in equation (9) and (10), one could use the self-attention mechanism resulting in a graph attention network (GAT) [41].

B. Factor Graph Neural Network-based Detection

A less complex variant of the GNN is the factor graph neural network (FGNN) proposed in [22]. The key difference is that the nodes only calculate the aggregation function and omit the MLP processing shown in equation (9) and equation (10). While this reduces the expressive potential of the node functions, the FGNN was shown to recover the max-log SPA and a low-rank approximation of it. The other, smaller difference is the consideration of the edge attributes. In a first step the node states are processed by a MLP unaware of the edge attributes. Secondly, the output is multiplied with a feature extraction matrix specific to the edge, which is generated by a MLP. This results in message update functions

$$\begin{aligned} \mathbf{m}_{V_i \rightarrow F_j} &= f_{\theta_E}(\mathbf{g}_{V_i \rightarrow F_j}) f_{\theta_{V \rightarrow F}}(\mathbf{s}_{V_i}, \mathbf{s}_{F_j}), \\ \mathbf{m}_{F_j \rightarrow V_i} &= f_{\theta_E}(\mathbf{g}_{V_i \rightarrow F_j}) f_{\theta_{F \rightarrow V}}(\mathbf{s}_{F_j}, \mathbf{s}_{V_i}), \end{aligned}$$

where f_{θ_E} is a MLP with weights θ_E generating the feature extraction matrix. In [22], it is mentioned that an MLP can be added to the nodes to increase expressiveness. Here, we do not do that, since the difference to the GNN would be practically non existent.

C. Channel Agnostic Embedding

Previously, we used the linear embedding function $\mathbf{s}_{F_i} = \mathbf{W} \cdot [\text{real}(y_i), \text{imag}(y_i)]^T$ from the channel outputs $\mathbf{y}$. However, the learned embedding matrix $\mathbf{W}$ only implicitly depends on the CIR $\mathbf{h}$. If $\mathbf{h}$ is constant during training, $\mathbf{W}$ can overfit to the CIR, but fails to work for varying CIR. The goal is to have an embedding function that is aware of $\mathbf{h}$ and processes the observations $\mathbf{y}$ accordingly. Here, we propose three solutions to the problem.

a) *Embedding of Log-Likelihoods*: The first solution is feeding channel agnostic log-likelihoods to the GNN. Simplifying equation (3), the M^{L+1} log-likelihoods per observation y_i for the FFG can be calculated by

$$\log(P(y_i | \tilde{x}_i, \dots, \tilde{x}_{i-L}, \mathbf{h})) = -\frac{\|y_i - \sum_{l=0}^L h_l \tilde{x}_{i-l}\|^2}{2\sigma^2}.$$

Since the log-likelihoods are a sufficient statistic and contain implicitly the complete knowledge of the CIR, using them is optimal but infeasible for large M or L . For the UFG, initializing the VNs of the GNN with equation (4) scales linear in M .

b) *Neural Channel State-aware Embedding*: The second solution is replacing the embedding matrix $\mathbf{W}$ by a small MLP where the CIR $\mathbf{h}$ is an input

$$\mathbf{s}_{F_i} = f_{\theta_E}(\text{real}(y_i), \text{imag}(y_i), \text{real}(\mathbf{h}), \text{imag}(\mathbf{h})).$$

The idea is that f_{θ_E} can approximate the log-likelihood $\log(P(y_i | \tilde{x}_i, \dots, \tilde{x}_{i-L}, \mathbf{h}))$ with low complexity by a d -dimensional embedding vector. Especially for a high SNR, $\log(P(y_i | \tilde{x}_i, \dots, \tilde{x}_{i-L}, \mathbf{h}))$ is sparse in terms of entries greater than 0. Intuitively, either the observation is close to a symbol combination and is highly likely, or the likelihood vanishes. Since most symbol combinations lay directly upon the observation, the result is a sparse likelihood. This facilitates the intuition that a low-dimensional approximation can be learned efficiently. Note, for the UFG y_i can be replaced by χ_i .

c) *Constant Channel Transformation*: The third idea is to transform the varying channel matrix $\mathbf{H}$ to a constant channel $\tilde{\mathbf{H}}$ via the constant channel transformation (CCT) receive filter [42]

$$\mathbf{R} = \tilde{\mathbf{H}} \left(\mathbf{H}^H \mathbf{H} + \frac{1}{\sigma^2} \mathbf{I} \right)^{-1} \mathbf{H}^H. \quad (11)$$

If $\tilde{\mathbf{H}}$ is constructed as in [43] and the memory of $\tilde{\mathbf{H}}$ is smaller than the memory of $\mathbf{H}$, then this procedure is called *channel shortening*. Here, we do not desire a smaller memory, but a constant $\tilde{\mathbf{H}}$ which, subsequently, can be detected by the GNN. As all operations in equation (11) are differentiable, $\tilde{\mathbf{H}}$ can be directly learned during training. $\tilde{\mathbf{H}}$ is initialized as a zero matrix with a band diagonal of ones with size $L + 1$. Finally, the new input to the embedding layer is $\tilde{\mathbf{y}} = \mathbf{R}\mathbf{y}$. This method scales well, as the complexity is independent on the memory L or the alphabet size M , but is sub-optimal. The transformation is only optimal for Gaussian inputs and an optimized choice of $\tilde{\mathbf{H}}$ based on $\mathbf{H}$. Here, neither is the case.

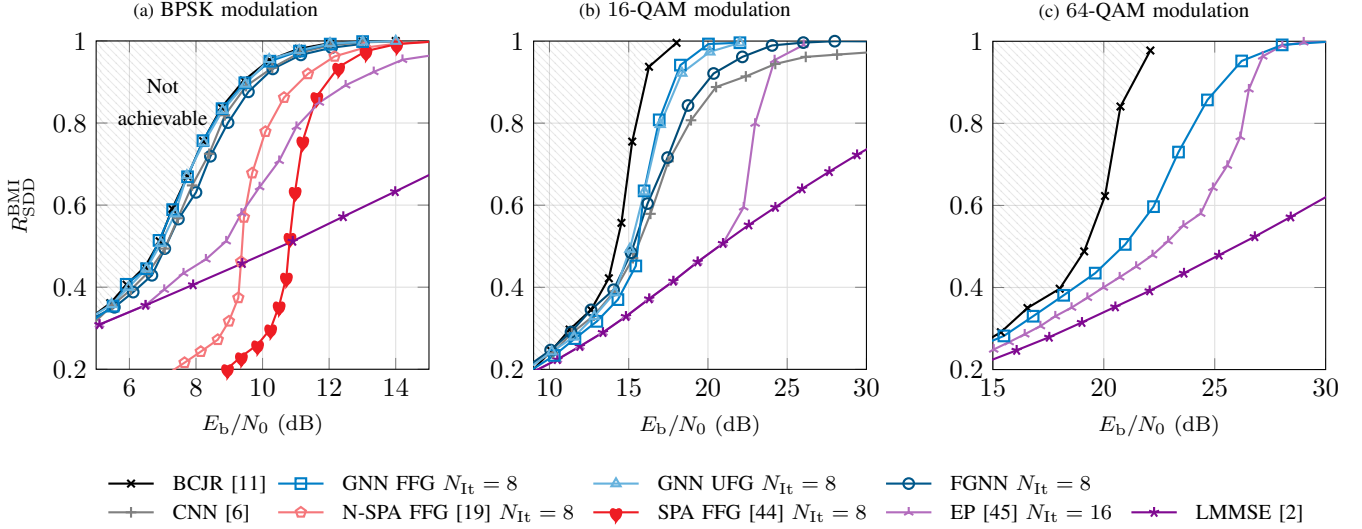


Fig. 4: Achievable rate of the GNN variants and various baselines over the Proakis-C channel. Note that, due to complexity the 16/64-QAM BCJR curve uses an equivalent 4/8-PAM. The GNN performs in the gap between APP-based and LMMSE-based detection outperforming the other NN-based approaches. For 64-QAM most NN-based approaches did not converge during training.

D. Training

For training, the BCE loss $\mathcal{L}_{\text{BCE}}$ is employed

$$\mathcal{L}_{\text{BCE}} = \frac{1}{N} \sum_{i=1}^N [c_i \log_2 q_i + (1 - c_i) \log_2 (1 - q_i)].$$

where c_i is the transmitted bit and q_i its estimate. Minimizing $\mathcal{L}_{\text{BCE}}$ is equivalent to maximizing the BMI $R_{\text{SDD}}^{\text{BMI}}$ (see equation (7)). In [46] it was shown that in practice, minimizing $\mathcal{L}_{\text{BCE}}$ minimizes the bit error rate (BER). Furthermore, the BCE loss is used in a multi-loss fashion, meaning it is averaged over all N_{It} GNN iterations $t \in [1, N_{\text{It}}]$ as

$$\mathcal{L}_{\text{Multi}} = \frac{1}{N_{\text{It}}} \sum_{t=1}^{N_{\text{It}}} \mathcal{L}_{\text{BCE}}^{(t)}. \quad (12)$$

This loss was first proposed in [20] and applied for GNN-based decoding in [24]. In addition, we use the Adam [47] optimizer.

The training data is artificially generated by the system shown in Fig. 1. The bits $\mathbf{u}$ are uniformly randomly generated and transmitted over the simulated channel to serve as input data to the GNN. Algorithm 2 summarizes the training procedure.

E. Results

Setup: For the evaluation of the proposed GNN detector, a transmission of $N_{\text{x}} = 512$ BPSK, 16-QAM and 64-QAM symbols over the Proakis-C channel ($\mathbf{h} = [0.227, 0.460, 0.688, 0.460, 0.227]$) are simulated. The channel is chosen to represent a scenario with severe ISI, where BPSK signaling results in at least an estimated loss of 2.01 dB and 5.01 dB compared to an AWGN channel, for JDD and SDD, respectively [48]. Note, the channel contains notches in the frequency representation posing a particular challenge for channel inverting detection, e.g., LMMSE detection. If not

Algorithm 2: Training epoch

Input : θ // Trainable parameters
Output: θ' // Updated train. parameters

// Data generation
 $\mathbf{u} \leftarrow \text{random_uniform_integer}(\{0, 1\}^K)$
 $\mathbf{y} \leftarrow \text{simulate_transmission}(\mathbf{u})$
 // GNN inference
 $\hat{\mathbf{u}} = \text{GNN_detector}(\mathbf{y}; \theta)$
 // Training
 $\mathcal{L}_{\text{Multi}} \leftarrow \text{Multi_loss}(\mathbf{u}, \hat{\mathbf{u}})$
 $\theta' \leftarrow \text{SGD}(\mathcal{L}_{\text{Multi}}, \theta)$

TABLE III: Hyperparameters of the GNNs and training

NN Parameter		Value	Train. Parameter	Value
Detection	# MLP hidden layers	2	Learning rate	10^{-4}
	# MLP hidden units	64	Batch size	256
	activation	ReLU	Epochs Det.	$5 \cdot 10^4$
	Feature size d	16	SNR Det.	10 dB to 14 dB
JDD	Schedule Flooding	(10, 1)	Epochs JDD	$1.6 \cdot 10^5$
	Schedule Sequential	(3, [3, 5])	SNR JDD	10 dB to 13 dB

stated otherwise, we assume a fixed channel. Therefore, the results demonstrate the algorithmic capability of GNN-based detection rather than generalization, robustness and adaptability aspects. The hyperparameters for the structure of the GNN and the training are shown in the top part of Tab. III. The parameters are based on [24]. We noticed that increasing the size (layers and units) of the MLPs compared to [24] enhances performance. Thus, we increased the parameters until we did not notice any more improvements in performance. For additional experiments regarding the training, we refer to the Appendix. As part of an ablation study, we investigated the use of GATs, i.e., self-attention for message aggregation. However, this does not seem improve the performance. An example is shown in the appendix. For all figures showing

achievable rates, each data points is optimized individually. That means retraining for NN-based detectors and multiplying the output with a damping factor for classical detectors [49].

Baselines: The GNNs are compared to several baselines that can be divided into APP-based, LMMSE-based, and NN-based detectors. The APP baselines are the following:

- The first APP-based detector is the BCJR algorithm [11], [50], implementing the optimal MAP estimator. However, the problem is exponential complexity $O(M^L)$ in the number of bits per symbol $\log_2(M)$ and channel memory L . In the case of a 16-QAM and the Proakis-C channel, the number of state transitions is $M^L \geq 10^6$.
- The second is the iterative SPA on the FFG and UFG. On the Proakis-C channel, the SPA on the UFG does not converge. Furthermore, SPA on the FFG exhibits the same exponential complexity as BCJR detection. While in general, more iterations result in better performance, the cycles in the factor graphs can result in an unstable behavior. Therefore, we apply a constant damping factor of 0.38 to all edges. This allows performing more iterations ($N_{It} = 11$) before divergence and, thus, significantly improves the performance. Without the damping factor, SPA diverges after $N_{It} = 5$ iterations.
- Better than guessing a constant damping factor is learning the damping weights per edge and per iteration, which is called neural sum-product algorithm (N-SPA) [19]. However, sharing the stability issues of SPA, the training of the N-SPA does not converge for $N_{It} > 7$.

Also, two LMMSE-based baseline schemes are shown:

- The first is LMMSE filtering [2] with subsequent memoryless demapping.
- The second is called expectation propagation (EP) [45], consisting of iterative message passing between a LMMSE filter and a demapper for memoryless channels.

Note that a combination of an optimal channel shortening filter and a BCJR detector is discussed in [43]. We observed that the performance of this combination is usually between the LMMSE and the EP detector, thus, providing only little insight for our study. Finally, we compare to a NN-based baseline:

- The CNN detector [6] can be interpreted as a trainable filter with subsequent APP processing, e.g., an optimized channel shortening filter and an APP equalizer with a small or no memory. The CNN consists of multiple identical groups of layers, so-called *blocks*. It is compared with $B = 3$ blocks for BPSK and $B = 6$ blocks for QPSK, 16-QAM and 64-QAM. Note that for QPSK (with memory $L = 6$) and 16-QAM, we found that increasing the number of filters per layer by a factor of 8 is necessary to ensure competitive performance.

Description BPSK, 16-QAM and 64-QAM Rate: Fig. 4a shows the achievable SDD rate of the GNN detectors based on the FFG and UFG, the FGNN based on the FFG, and several baselines. The other NN-based detectors (FGNN and CNN) slightly deteriorate for medium and high rates from the GNNs by a gap of roughly 0.5 dB. The LMMSE-based baselines (LMMSE and EP) perform close to the optimal MAP

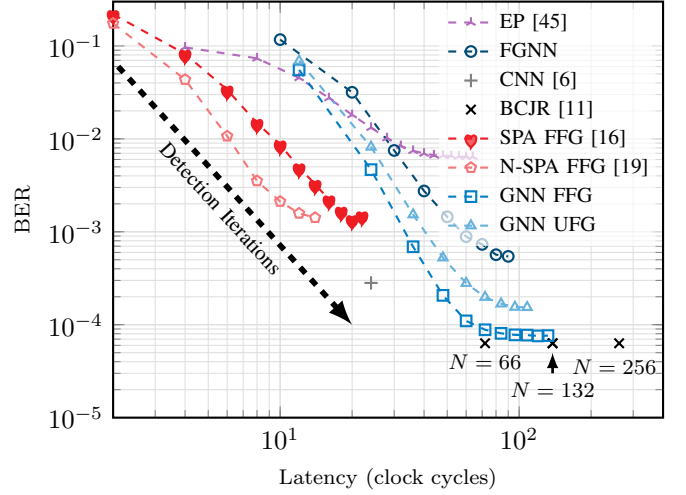


Fig. 5: BER versus latency using BPSK at SNR 14 dB of different detectors over the Proakis-C channel. For iterative detectors, each marker corresponds to the performance after an iteration.

performance of the BCJR detector for low rates, however, display sub-optimal behavior for moderate and high rates.

Fig. 4b shows the achievable SDD rate using a 16-QAM. Again, a BCJR baseline displays the MAP performance. However, as the 16-QAM is computationally infeasible, a 4-PAM is used as surrogate which has identical performance for this special case where the CIR is real-valued $\mathbf{h} \in \mathbb{R}$. For the GNNs and other baselines, a 16-QAM is used. The neural approaches (GNNs, FGNN and CNN) perform better than the classical approaches. For high rates the GNN based on the FFG performs best. Finally, (neural) SPA is not shown, due to infeasible complexity or non convergence for FFG and UFG, respectively.

Fig. 4c displays the result for 64-QAM. Here, the GNNs on the FFG show slight performance gains compared to EP, however, at a increased complexity. This indicates a limited scalability of the GNNs in terms of modulation order. The other NN-based approaches did not converge during training.

Latency: Fig. 5 shows the BER vs. latency in clock cycles. We assume that every node or neural network layer is processed in one clock cycle. This means 12 cycles per iteration for the GNNs, 10 for the FGNN, 4 for EP, and 2 for (neural) SPA. The latency of the BCJR detector equates to $N_x + L + 2$, and is, thus, dependent on the block length. We assume a parallel computation of the forward and backward metric in $N_x + L$ cycles and 2 additional cycles are needed to calculate the channel likelihoods and the bit estimates. Consequently, inherent parallelism of the GNN shows larger latency gains over the BCJR as the transmission length increases.

F. Analysis

This section analyses how GNNs might improve over their basis, the SPA. Note that this is not an analysis in the sense of explainable GNNs as in [51]. Fig. 5 reveals two advantages of the GNN compared to the classical SPA or neural SPA counterparts. The first is the lower BER after a single iteration, i.e., the first data point of the SPA variants is higher than the

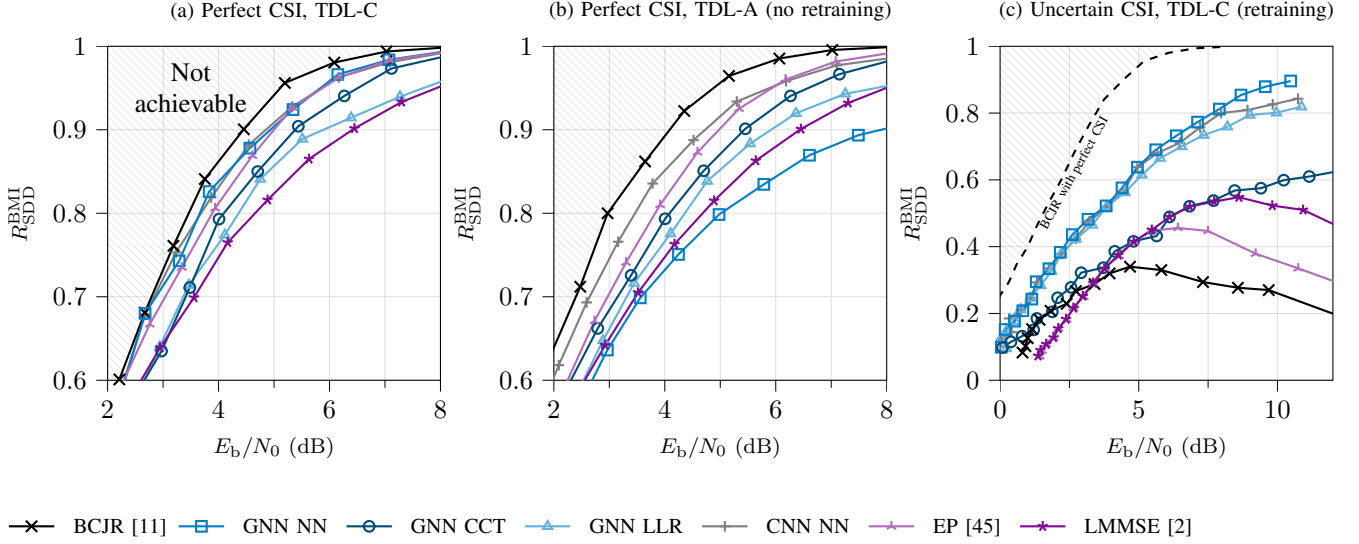


Fig. 6: Achievable rate of the GNN variants and various baselines using QPSK modulation trained over a 5G TDL-C channel (7 taps) with the channel embedding strategies: NN, CCT, and LLR. Note, $\mathbf{h}$ is random for every transmission but follows a specified distribution. All detectors use $N_{\text{It}} = 8$ iterations. Uncertain CSI means detection based on a noisy CIR $\mathbf{h}' = \mathbf{h} + \mathbf{n}'$ where $\mathbf{n}' \sim \mathcal{N}(\mathbf{0}, 0.15\mathbf{I})$. The NN-based detectors are robust against CSI uncertainty, but potentially degrade, if channel changes.

first GNN data point. The second is the larger decrease in BER per iteration, as the slope of the GNN is steeper. The reason for the better initial BER is the construction of sub-optimal messages in the first iteration by the (neural) SPA algorithm. The result of VN V_j is the marginalization $\prod_{i=j-L}^j P(x_j|y_i)$. However, the optimal decision is

$$\begin{aligned}
 P(x_j|y_j, \dots, y_{j-L}) &= p(y_j, \dots, y_{j-L}|x_j) \frac{P(x_j)}{p(y_j, \dots, y_{j-L})} \\
 &\propto p(y_j|x_j) \cdot p(y_{j-1}|y_j, x_j) \cdot \dots \\
 &\quad \cdot p(y_{j-L}|x_j, y_i, \dots, y_{i-L}) \\
 &\propto P(x_j|y_j) \cdot \frac{p(x_j, y_j|y_{j-1})}{p(x_j, y_j)} \cdot \dots \\
 &\quad \cdot \frac{p(x_j, y_{j-1}, \dots, y_{j-L+1}|y_{j-L})}{p(x_j, y_{j-1}, \dots, y_{j-L+1})} \\
 &= P(x_j|y_j) \cdot P(x_j|y_{j-1}) \cdot \underbrace{\frac{p(y_j|y_{j-1}, x_j)}{p(x_j, y_j)}}_{\text{omitted in SPA}} \cdot \dots \\
 &\quad \cdot P(x_j|y_{j-L}) \cdot \underbrace{\frac{p(y_j, \dots, y_{j-L+1}|y_{j-L}, x_j)}{p(x_j, y_{j-1}, \dots, y_{j-L+1})}}_{\text{omitted in SPA}}.
 \end{aligned}$$

We observe that the messages for an optimal decision contain information about the dependencies between the observations $y_i, \dots, y_{i-L}$, which are omitted in the SPA. Meaning the SPA messages in the first iteration are only optimal if the observations are independent. This is not the case for channels with memory resulting in an sub-optimal decision. As a consequence, the messages of the GNN, which are abstract feature vectors, could capture the inter-dependencies of $y_i, \dots, y_{i-L}$ leading to improved decisions early on. Combining improved messages with adaptive damping may explain the larger decrease in BER per iteration. In [52] it was demonstrated that adaptive damping of messages can be beneficial compared to fixed damping. Here, adaptive means that the damping factor

is based on the message, which could be learned by the MLPs in the GNN. These intuitions and the line of reasoning is further supported by the following experiment. We trained four variants of the GNN on the FFG: the full GNN as described, the GNN but with a summation as VN, the GNN but with a $\max^*$ as FN, and the GNN but with equation (5) as edges. Each replacement is the classical counterpart in the SPA. We observe that only the variant with the classical SPA calculation on the edges deteriorates in performance. Meaning that the MLP on the edge improves the performance complementary to the adaptive scaling. Further, the adaptive scaling, as in [52], does not need to happen in the FN, but can also be implemented in the VN or on the edge. However, a trainable FN remains beneficial in terms of complexity on the FFG as it may not require exponential complexity in M or L . Finally, the VN appears to not benefit from a MLP in terms of performance nor complexity. Thus, a summation as the VN update suffices.

G. Robustness and Adaptivity

One key aspect to analyze of a NN-based detector is the robustness [53], i.e., the robustness to uncertainty in CSI or a distribution change of the CIR. Fig. 6 depicts results based on the proposed channel agnostic interfaces, which are based on LLRs, NN, and a constant channel transformation. The channel is a 5G TDL-C model [54] with 20 MHz bandwidth, 100 ns delay spread, and 7 taps. We evaluate three different scenarios:

- 1) Perfect CSI, where training and inference are performed on the same channel model.
- 2) Perfect CSI, where training is performed on TDL-C and inference on TDL-A. This scenario captures the change in distribution of the CIR. Note that the baselines are not negatively influenced by the change as TDL-A (with these parameters) allows for higher rates than TDL-C.
- 3) CSI uncertainty: A noisy CIR is passed to the detector. The noisy CIR is constructed by $\mathbf{h}' = \mathbf{h} + \mathbf{n}'$ where

$\mathbf{n}' \sim \mathcal{N}(\mathbf{0}, 0.15\mathbf{I})$. Note that the GNNs are retrained for this case.

For the baselines we observe that the better the performance with genie CSI, the worse it performs with noisy CSI. For the GNNs variants there seems not to be a clear trend. The NN-based channel interface performs the best in the first scenario, but suffers the most in a change in distribution (second scenario) of the CIR. However, it is the best model if retrained for the case of CSI uncertainty. This means that the NN-based interface can adapt to scenario changes if retrained, but may be vulnerable if not. In general, the NN-based detectors outperform all classical baselines in the case of CSI uncertainty. Similar to the LLR-based variant, the CCT is robust in terms of changes in CIR distribution. However, the transformation bottlenecks the performance in case of CSI uncertainty, as the GNN cannot restore the lost information. It is worth noting that the LLR-based interface performs the worst in the first scenario. This is probably due to the embedding, since the LLR vector contains 16384 values which are projected onto the feature space of the GNN of size $d = 16$.

IV. GRAPH NEURAL NETWORKS FOR JOINT DETECTION AND DECODING

In this section, we describe an NN for JDD and TDD based on GNNs. For evaluating systems with feedback from a decoder, the appropriate performance metric is either the BER or the block error rate (BLER). A fully deep learning-based receiver allows to jointly optimize the component networks rather than an individual optimization like in traditional receivers. This is called end-to-end learning. We see the potential of this receiver mostly in the short block length regime, where sub-optimal SPA decoding leaves room for improvement. For longer block lengths, we propose to use the GNNs as the detection component in TDD. Note that in this case, the detector is trained independently of the channel code, but assumes *a priori* knowledge. As a result, the appropriate evaluation metric for detection is the iterative achievable rate shown in equation (8). Furthermore, using GNNs for the components prevents the NNs from the *curse of dimensionality* (referring to exponential growth of points in high-dimensional objects) [55], since the graph-based structures leverage the sparseness of the connections. Moreover, the GNNs use high-dimensional processing the nodes and edges, leveraging the *blessing of dimensionality* [56].

The structure of the GNN for JDD is displayed in Fig. 7. One set of FNs is based on the graph for detection as in Sec. III and one of FNs for decoding as in [24]. Note that both graphs share the same VNs, building a connected graph. In the case of using the GNN as a detector in TDD, an embedding of the *a priori* LLRs ℓ_A to the VNs is proposed. Similar to the embedding of the FNs we propose a linear embedding $\mathbf{s}_{V_i} = \mathbf{W}_V \cdot \ell_A$. Furthermore, interleaving and deinterleaving makes the *a priori* information of the detector and decoder to appear more independent, thus, improving performance [57].

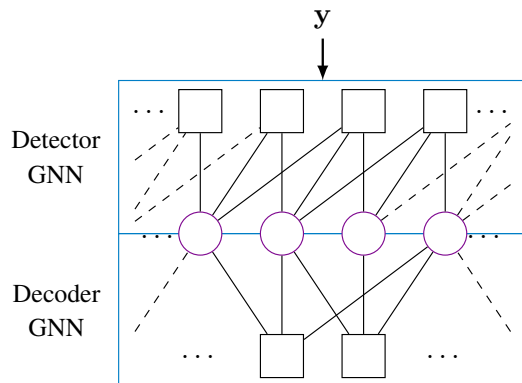


Fig. 7: JDD system with a decoder GNN based on the Tanner graph of the channel code and an detector GNN based on the FFG of the channel. Note that the Tanner graph of the channel code has been rearranged according to the interleaver to directly match the detector VNs.

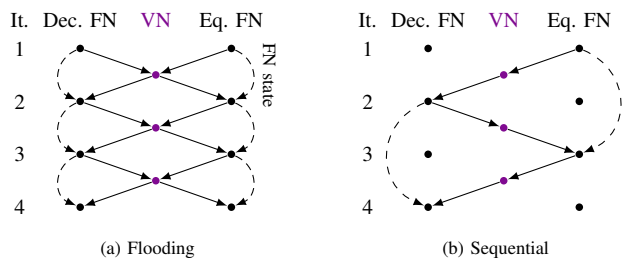


Fig. 8: Schedules of the GNN-based JDD

A. Schedule

In contrast to the classical solution of using iterative BCJR detection and SPA decoding [58], the combined GNN for joint detection and decoding enables an earlier use of detection with *a priori* knowledge. BCJR runs are costly in complexity and latency, thus, a sufficient amount of iterations should be spend on SPA decoding to generate a significant amount of *a priori* information for the subsequent BCJR run. However, during this time, SPA decoding does not benefit from new *a priori* information from the detector. In case of the combined GNN, the latency for detection and decoding are the same, thus, enabling schedules with earlier information exchange between the components. This can result in improved error correcting performance at the same latency. Moreover, the combined GNN can be updated in a flooding fashion. This means that all FNs (for detection and decoding) are updated in parallel and all VNs are updated in parallel. The difference to the iterative, sequential schedule is shown in Fig. 8. The GNN updates the FNs for detection and for decoding simultaneously and combines their beliefs after each iteration in their shared VNs. Relating the description to Algorithm 1, the only addition is that the set of FNs also contains the check nodes of the LDPC code. Consequently, the same number of resources can be used as in the sequential, iterative case, but in half the time. In every iteration, the component GNNs receive new *a priori* information but also keep their respective FN state. In [59], it was found that keeping the state of the FNs is beneficial for iterative systems with a small number of inner iterations.

B. Training

The training is similar to the training process described in section III-D with some extensions. For the JDD GNNs based on the combined factor graph is trained at once. In the case of TDD with SPA decoder, we employ *training with Gaussian priors* [6], [60], [61]. Note that the 5G LDPC code contains punctured VNs that we include in the calculation of the loss. The additional hyperparameters are shown below the horizontal line in Tab. III. The schedule of the combined GNNs is given in the form ($\#$ Outer Iterations, [$\#$ Inner Iterations]). For deep GNNs, i.e., a large number of iterations, direct training led to sub-optimal performance. Therefore, we used a two stage training process in these cases. First, we trained with the schedule given in Tab. III, for which the GNNs trained reliable and consistent. Second, the number of iterations is increased to the desired schedule and the model is finetuned. A flexible schedule is ensured by the weight-sharing over iterations in the GNNs.

For the training with Gaussian priors the GNN for TDD is trained independently of the decoder, as shown in Algorithm 3. The process is similar to evaluation of the EXIT characteristic detailed in section II-D. Similarly, the priors are generated from equation (6) with a specified average mutual information I_A . The VNs of the GNN are initialized with the *a priori* LLRs. In contrast to the EXIT procedure, we are not interested in the extrinsic information I_E , but the total information I_T . The loss in equation (12) is then calculated between the transmitted bits and the output of the GNN. The extrinsic LLRs for inference can be calculated with equation (5). An additional hyperparameter of the training with Gaussian priors is the distribution of I_A . We choose $I_A \sim \mathcal{U}(0, 1)$, since the GNN should generalize for all possible prior information.

Algorithm 3: Gaussian pre-training epoch for TDD

Input : θ // Trainable parameters
Output: θ' // Updated train. parameters

```

// Data generation
 $\mathbf{u} \leftarrow \text{random\_uniform\_integer}(\{0, 1\}^K)$ 
 $\mathbf{y}, \mathbf{c} \leftarrow \text{simulate\_transmission}(\mathbf{u})$ 
// Gaussian prior generation
 $I_A \leftarrow \text{random\_uniform\_float}((0, 1))$ 
 $\ell_A \leftarrow \text{generate\_LLRs}(\mathbf{c}, I_A)$ 
// GNN inference
 $\hat{\mathbf{c}} = \text{GNN\_detector}(\mathbf{y}, \ell_A; \theta)$ 
// Training
 $\mathcal{L}_{\text{BCE}} \leftarrow \text{BCE\_loss}(\mathbf{c}, \hat{\mathbf{c}})$ 
 $\theta' \leftarrow \text{SGD}(\mathcal{L}_{\text{Multi}}, \theta)$ 

```

C. Results

JDD in the short block length regime: For showcasing the potential of GNNs for JDD, especially in low latency scenarios, we evaluate a transmission encoded by a 5G LDPC code ($K = 66, N = 132, R_C = 0.5$) and BPSK signaling over the Proakis-C channel. In Fig. 9, the BER of the bits $\mathbf{u}$ is shown over the SNR. The schedules are given in the legend

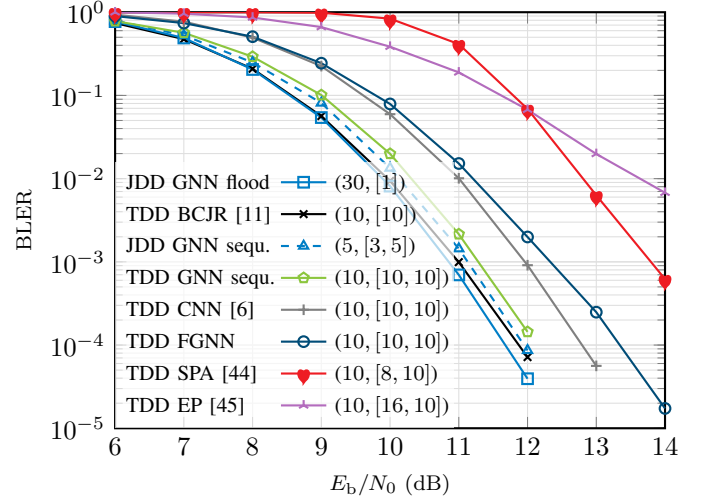


Fig. 9: BER of the bits $\mathbf{u}$ using coded system ($N = 132, R_C = 0.5$, 5G LDPC, BPSK) over SNR of different detectors and decoders over the Proakis-C channel. The JDD/TDD schedules are given in ($\#$ Outer Iterations, [$\#$ Inner Iterations]). All TDD systems use an SPA decoder. The flooding GNN outperforms the best TDD baseline.

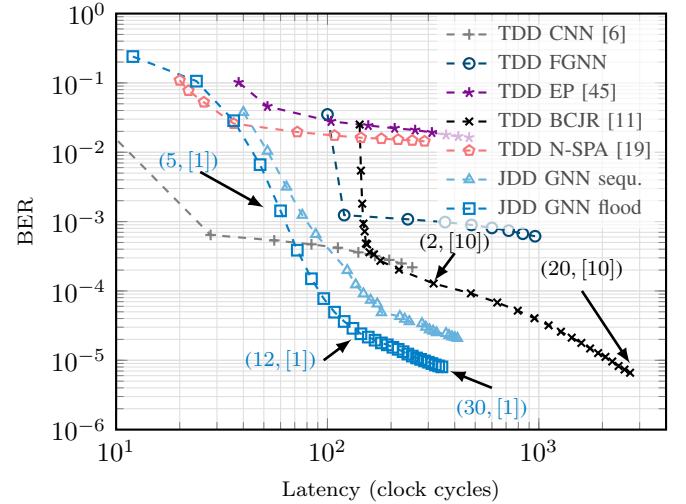


Fig. 10: BER of uncoded bits $\mathbf{u}$ using coded system ($N = 132, R_C = 0.5$, 5G LDPC, BPSK) versus latency at SNR 12 dB of different detectors and decoders over the Proakis-C channel. The flooding GNN approaches the TDD BCJR performance with 15% of the latency.

in the form ($\#$ Outer Iterations, [$\#$ Inner Iterations]). The flooding schedule GNN outperforms the TDD BCJR-SPA baseline. Unexpectedly, the flooding GNN outperforms the sequential GNN regardless of the latency constraints, suggesting that the flooding schedule may be advantageous during training. Finally, a GNN trained with Gaussian priors in a TDD framework is displayed. Regardless of the schedule, the JDD GNNs outperform the TDD GNN. Fig. 10 displays the BER over the latency in clock cycles. Recall that 12, 2, and $N + L + 2$ clock cycles are needed for the flooding GNN, SPA and BCJR, respectively. In the low latency regime, the flooding GNN outperforms the sequential GNN and the BCJR-SPA baseline, as it demonstrates a BER three times lower than the sequential GNN and ten times lower than the BCJR-SPA baseline. The flooding GNN is only outperformed by the BCJR-SPA baseline after ca. seven times the latency of the GNN.

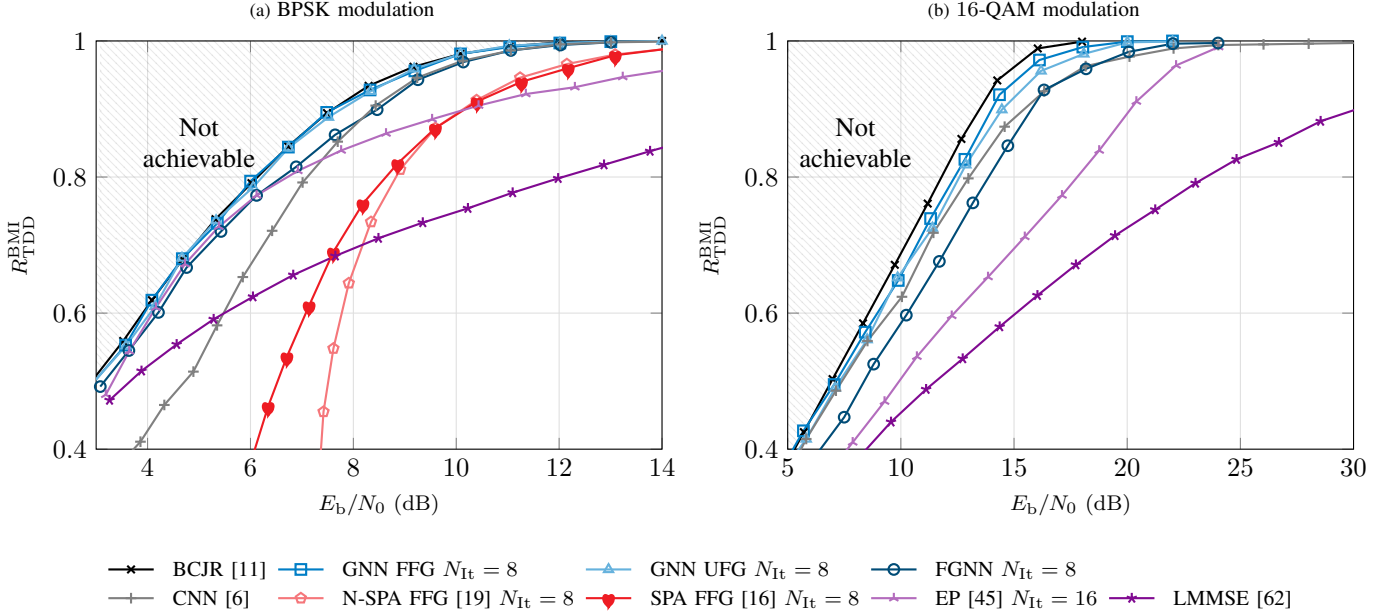


Fig. 11: Achievable rate over the SNR for TDD and 16-QAM modulation over the Proakis-C channel. Note, due to complexity the 16-QAM BCJR curve uses an equivalent 4-PAM. The GNN performs in the gap between APP-based and LMMSE-based detection outperforming the other NN-based approaches.

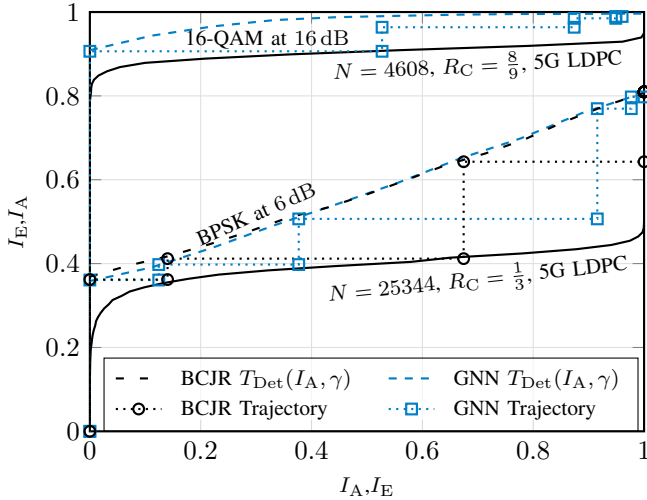


Fig. 12: EXIT chart for a high rate scenario ($N = 4608$, $R_C = \frac{8}{9}$, 5G LDPC with $N_{It} = 10$, 16-QAM) and a low rate scenario ($N = 25344$, $R_C = \frac{1}{3}$, 5G LDPC with $N_{It} = 10$, BPSK) over the Proakis-C channel. The GNN trajectory matches the prediction from the transfer characteristic $T_{Det}(I_A, \gamma)$.

TDD in the long block length regime: In the following we demonstrate the viability of GNNs for detection in TDD. Fig. 11a shows the achievable rate with TDD (equation (8)) compared to several baselines. The result is similar to the 16-QAM scenario depicted in Fig. 11b, specifically, that the GNNs perform close to the optimum. Especially in the high rate regime, NN-based detection seems to be beneficial to the non-NN baselines.

The validity of the rate is evaluated in the EXIT chart depicted in Fig. 12. The trajectory is transfer characteristic is shown for two cases. The first case is a high rate case with a 5G LDPC code ($N = 4608$, $K = 4032$, $\frac{E_b}{N_0} = 16$ dB) and 16-QAM. The trajectory does not match the EXIT

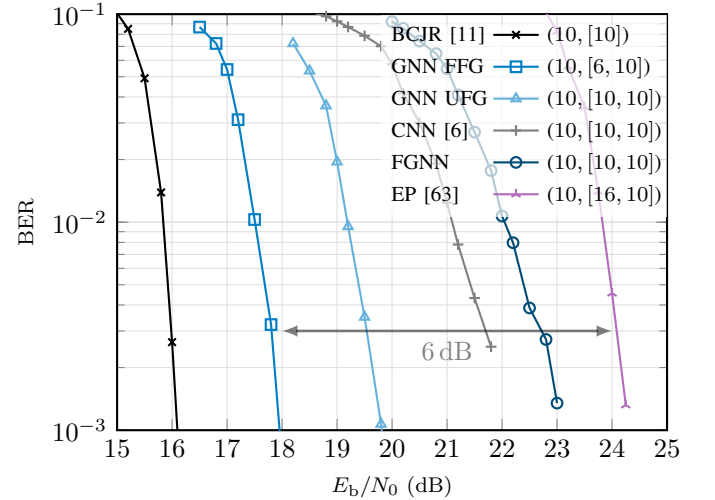


Fig. 13: BER of the bits $\mathbf{u}$ using coded system ($N = 4608$, $R_C = \frac{8}{9}$, 5G LDPC, 16-QAM) over SNR of different detectors and decoders over the Proakis-C channel. The JDD/TDD schedules are given in (#Outer Iterations, [#Inner Iterations]). Note that, for complexity reasons, the BCJR curve uses an equivalent 4-PAM. The GNN shows a gain of 6 dB compared to EP, the best and feasible classical baseline.

characteristic precisely. A potential issue may be that the extrinsic LLRs from the decoder do not resemble Gaussian distributions, compared to the Gaussian priors during training. The second case is a low rate 5G LDPC code ($N = 25344$, $K = 8448$, $\frac{E_b}{N_0} = 6$ dB) and BPSK. In the second case, the BCJR trajectory and transfer characteristic is shown for comparison. Here, the EXIT characteristic is matched more closely. However, a small divergence occurs after three turbo iterations. Note that the divergence is after decoding, meaning the *a priori* information from the GNN might be not extrinsic. This problem may be attributed to the loops in the graph. This aligns with the observation that equation (5) (which is used in

TABLE IV: Overview of detection schemes and categorization of properties into: good (+), mediocre (o), and bad (-).

Name	Scalability	Latency	Severe ISI	Type	High rates	Reference
SPA-UFG	+	+	-	APP	o	[17], [19]
SPA-FFG	-	+	o	APP	+	[19], [44]
BCJR	-	-	+	APP	+	[11], [50]
LMMSE	+	+	-	LMMSE	-	[2], [62]
EP	+	o	o	LMMSE & APP	+	[45], [63]
CNN	o	+	+	Neural	+	[6], [7]
GNN	o	+	+	Neural	+	proposed

TDD) does not hold for GNNs, if $R_{\text{SDD}} \ll I_A$. In this case the *a priori* LLRs dominate inference and the loops lead to too significant amplification.

Finally, we evaluate the GNNs in the TDD framework in the high rate scenario ($N = 4608$, $K = 4032$) and 16-QAM in Fig. 13. Recall that the BCJR has more than 10^6 state transitions and is, thus, infeasible. Compared to the best, feasible non-NN baseline, the GNN based on the FFG demonstrates a gain of more than 6 dB.

V. DISCUSSION

Before we discuss how NN-based detection, in particular GNNs, fit in to the landscape of detection, we detail the areas where classical detection leaves room for improvement. A summary is presented in Tab. IV.

Let us start with the ideal case of genie CSI. While the APP-based approaches (BCJR, SPA on FFG) scale exponentially, the complexity for small alphabet size M and memory L , i.e. $M^L \lesssim 10^2$ is relatively small. Thus, only for $M^L \gtrsim 10^2$ alternatives are required. The complexity of LMMSE-based detection is almost independent of M and L , making them promising candidates. The matrix inversion of complexity $\mathcal{O}(N^3)$ (or $\mathcal{O}(N \log(N))$ for Toeplitz matrices [64]) can be avoided by a filter-type implementation. This can be a time-invariant filter for LMMSE detection or a time varying filter for EP [63]. However, LMMSE detection performs highly sub-optimal for high rates. Furthermore, EP improves LMMSE detection for high rates on the cost of iterative LMMSE filtering with increased complexity of $\mathcal{O}(N^2)$ [65]. The first problem with LMMSE-based approaches are notches in the frequency representation of the channel, as the LMMSE tries inverting the channel. The second problem is $p(\mathbf{y}, \mathbf{x})$ being non-Gaussian. The probability of the first issue decreases with increased channel memory L (for stochastic CIRs), since the probability of destructive interference decreases. The second issue can vanish for increasing M assuming Gaussian-like constellation shaping. To summarize, NN-based detection may be beneficial if M and L are too large for APP-based detection, but small enough so that $p(\mathbf{y}, \mathbf{x})$ is not yet Gaussian, and the CIR has notches in the frequency domain.

Another area where NN-based detection may shine, are imperfections. This could be a non-linear channel as in [6], [10] or imperfect CSI at the receiver as shown here or discussed in more detail in [12], [53]. These imperfections are usually hard to consider in a classical detection algorithm, but can be easily learned by an NN.

The last area worth mentioning is the possibility of a end-to-end optimized receiver. Instead of optimizing each component in the receiver it might be beneficial to optimize them jointly,

which is enabled by the use of NNs. This is demonstrated here for JDD, and also considered in, e.g., [26], [66]–[68].

The performance gains promised by NN-based receivers, here GNNs, may come with increased complexity compared to LMMSE-based approaches. However, quantifying the complexity is not trivial. A meaningful comparison can only be conducted on an implementation level, due to the different type of functions and operations used. For this, one would first need to optimize the NN size (e.g., by pruning), quantize it, and choose a suitable implementation and hardware [69], [70]. Here, all the NNs (GNN, CNN, and FGNN) are over parameterized and hence, showing the performance limit. Nonetheless, for an approximate complexity comparison between the NN-based approaches and classical detectors, we assume that they are executed on a general hardware, e.g., GPU or TPU. Then, the metric of interest is the number of weights and the number of operations [19], as shown in Tab. V. The operation count of the BCJR and SPA-UFG is taken from [19]. Similar to BCJR detection, the operation count of SPA on the FFG amounts to $N_{\text{It}}(M^{L+1}(L+2) + LM(2L+1))$, where L is the memory and M is the modulation order. Next, the GNN requires $N_{\text{It}}2(2+L)((N_L-2)N_U(N_U+2) + 4N_Ud)$ operations, where N_L is the number of layers in each MLP, N_U is the number of units in each MLP and d is the feature size. Finally, we approximate counting the operations of EP by counting the operations of the dominating LMMSE step. Disregarding simplifications in the matrix inversion, the filter-type implementation from [63] yields $72L^3 + 60L^2 + 96L + 10 + N_{\text{It}}(45L^3 + 66L^2 + 34L + 6)$ operations.

TABLE V: Rounded number of operations (multiplications, additions, activations, max*) for different modulation orders M and channel memories L for $N_{\text{It}} = 8$ iterations for EP, SPA and the GNN.

Algorithm	QPSK ($M = 4$)		16-QAM ($M = 16$)		# weights
	$L = 4$	$L = 6$	$L = 4$	$L = 6$	
BCJR	$2 \cdot 10^3$	$7 \cdot 10^4$	$7 \cdot 10^7$	$2 \cdot 10^{10}$	-
EP	$4 \cdot 10^4$	$1 \cdot 10^5$	$4 \cdot 10^4$	$1 \cdot 10^5$	-
SPA-FFG	$5 \cdot 10^4$	$1 \cdot 10^5$	$5 \cdot 10^7$	$2 \cdot 10^{10}$	-
SPA-UFG	$1 \cdot 10^3$	$2 \cdot 10^3$	$2 \cdot 10^4$	$4 \cdot 10^4$	-
GNN	$6 \cdot 10^5$	$8 \cdot 10^5$	$6 \cdot 10^5$	$8 \cdot 10^5$	$2 \cdot 10^4$
CNN	$5 \cdot 10^5$	$5 \cdot 10^5$	$5 \cdot 10^5$	$5 \cdot 10^5$	$5 \cdot 10^5$

VI. CONCLUSION

The paper proposes and demonstrates the application of GNNs for detection and JDD. In the case of detection, the GNN based on the FFG shows close-to-optimal performance. Thus, GNNs may be regarded as an elegant way of bridging the performance gap between SPA and BCJR for detection. Especially for high rates, the GNN outperforms state-of-the-art baselines. We also provided some intuition and reasoning that

the gains might come from improved message passing between the nodes. In the case of JDD, we demonstrated the end-to-end learning capabilities of GNNs for detection and decoding, leveraging sparse graphs and the blessing of dimensionality. With the proposed flooding schedule, the GNN outperforms an iterative BCJR-SPA baseline with a substantially lower BER. Furthermore, we investigated the GNN in a TDD setup, where a gain of more than 6 dB was demonstrated.

APPENDIX

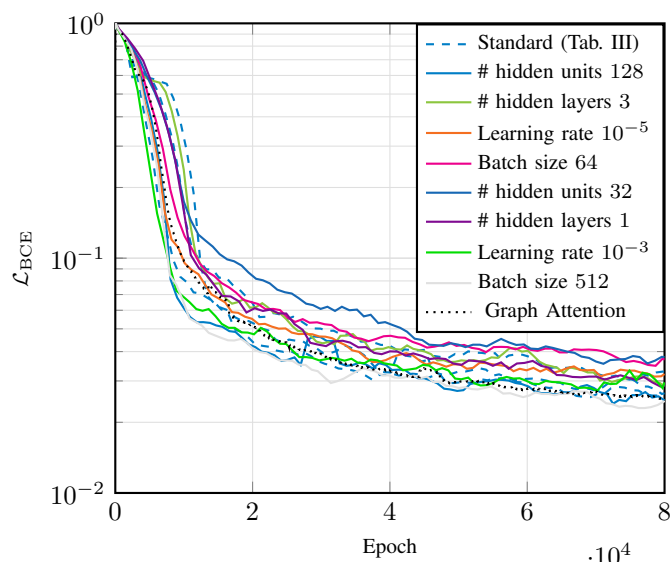


Fig. 14: Evaluation loss over epochs for JDD training over the Proakis-C channel at 10 dB with BPSK modulation and a 5G-LDPC code ($N = 132$, $R_C = 0.5$). The GNN is based on the FFG and $N_{It} = 10$

In this section, we provide additional data regarding the training process. Fig. 14 shows the smoothed evaluation loss over the training epochs for a JDD scenario for several experiments. The experiments vary the training hyperparameters and the structure hyperparameters. Furthermore, several training runs with the parameters from Tab. III are shown, indicating a stable and reproducible result. The differences in the experiments are displayed in the legend by showing which parameter changes in contrast to Tab. III. Overall, we observe that alterations of the parameters lead to only small variations in the training process. A similar amount of variation is observed by replicating the training with the parameters from Tab. III, but with different initiations. Finally, we conclude that the training process is stable, reproducible and is only mildly influenced by the choice of hyperparameters.

To explore the performance limits of the GNN approach, we performed additional experiments with GATs. As shown in Fig. 14, the best training run indicates that more sophisticated architectures may not necessarily improve performance.

REFERENCES

- [1] J. Clausius, M. Geiselhart, D. Tandler, and S. ten Brink, "Graph Neural Network-Based Joint Equalization and Decoding," in *2024 IEEE International Symposium on Information Theory (ISIT)*, 2024, pp. 1203–1208.
- [2] J. Proakis, *Digital Communications*. McGraw-Hill, 2001.
- [3] Xiaohu You et al., "Towards 6G wireless communication networks: vision, enabling technologies, and new paradigm shifts," *Science China Information Sciences*, vol. 64, Nov. 2020.
- [4] G. Gibson, S. Siu, and C. Cowen, "Multilayer perceptron structures applied to adaptive equalisers for data communications," in *International Conference on Acoustics, Speech, and Signal Processing*, 1989, pp. 1183–1186 vol.2.
- [5] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based Deep Learning," *Proceedings of the IEEE*, 2023.
- [6] X. Huang, J. Cho, K. Hashemizadeh, and R.-R. Chen, "Extrinsic Neural Network Equalizer for Channels with High Inter-Symbol-Interference," in *ICC 2021 - IEEE International Conference on Communications*, 2021, pp. 1–6.
- [7] W. Xu, Z. Zhong, Y. Be'ery, X. You, and C. Zhang, "Joint neural network equalizer and decoder," in *2018 15th International Symposium on Wireless Communication Systems (ISWCS)*, 2018, pp. 1–5.
- [8] G. Kechriotis, E. Zervas, and E. Manolakis, "Using recurrent neural networks for adaptive communication channel equalization," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 267–278, 1994.
- [9] N. Farsad and A. Goldsmith, "Neural Network Detection of Data Sequences in Communication Systems," *IEEE Trans. on Signal Process.*, vol. 66, no. 21, pp. 5663–5678, 2018.
- [10] D. Plabst, T. Prinz, F. Diedolo, T. Wiegart, G. Böcherer, N. Hanik, and G. Kramer, "Neural network-based successive interference cancellation for non-linear bandlimited channels," *IEEE Transactions on Communications*, pp. 1–1, 2024.
- [11] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv, "Optimal decoding of linear codes for minimizing symbol error rate (corresp.)," *IEEE Transactions on Information Theory*, vol. 20, no. 2, pp. 284–287, March 1974.
- [12] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, "Data-driven factor graphs for deep symbol detection," in *2020 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2020, pp. 2682–2687.
- [13] G. Forney, "Lower Bounds on Error Probability in the Presence of Large Intersymbol Interference," *IEEE Transactions on Communications*, vol. 20, no. 1, pp. 76–77, 1972.
- [14] G. Ungerboeck, "Adaptive Maximum-Likelihood Receiver for Carrier-Modulated Data-Transmission Systems," *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 624–636, 1974.
- [15] F. Kschischang, B. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Transactions on Information Theory*, vol. 47, no. 2, pp. 498–519, 2001.
- [16] G. Colavolpe and G. Germini, "On the application of factor graphs and the sum-product algorithm to ISI channels," *IEEE Transactions on Communications*, vol. 53, no. 5, pp. 818–825, 2005.
- [17] G. Colavolpe, D. Fertonani, and A. Piemontese, "SISO Detection Over Linear Channels With Linear Complexity in the Number of Interferers," *IEEE Journal of Selected Topics in Signal Processing*, vol. 5, no. 8, pp. 1475–1485, 2011.
- [18] B. Liu, S. Li, Y. Xie, and J. Yuan, "A Novel Sum-Product Detection Algorithm for Faster-Than-Nyquist Signaling: A Deep Learning Approach," *IEEE Transactions on Communications*, vol. 69, no. 9, pp. 5975–5987, 2021.
- [19] L. Schmid and L. Schmalen, "Low-Complexity Near-Optimum Symbol Detection Based on Neural Enhancement of Factor Graphs," *IEEE Transactions on Communications*, vol. 70, no. 11, pp. 7562–7575, 2022.
- [20] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *Allerton Conf.* IEEE, 2016, pp. 341–346.
- [21] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [22] Z. Zhang, M. H. Dupty, F. Wu, J. Q. Shi, and W. S. Lee, "Factor graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 181, pp. 1–54, 2023.
- [23] V. G. Satorras and M. Welling, "Neural Enhanced Belief Propagation on Factor Graphs," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2021, pp. 685–693.
- [24] S. Cammerer, J. Hoydis, F. A. Aoudia, and A. Keller, "Graph Neural Networks for Channel Decoding," in *IEEE Global Communications Conference (GLOBECOM) Workshop*, 2022.
- [25] A. Scotti, N. N. Moghadam, D. Liu, K. Gafvert, and J. Huang, "Graph Neural Networks for Massive MIMO Detection," *ArXiv*, vol. abs/2007.05703, 2020.
- [26] S. Cammerer, F. A. Aoudia, J. Hoydis, A. Oeldemann, A. Roessler, T. Mayer, and A. Keller, "A Neural Receiver for 5G NR Multi-User MIMO," *2023 IEEE Globecom Workshops (GC Wkshps)*, pp. 329–334, 2023.

- [27] A. Kosasih, V. Onasis, V. Miloslavskaya, W. Hardjawana, V. Andrean, and B. Vucetic, "Graph Neural Network Aided MU-MIMO Detectors," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 9, pp. 2540–2555, 2022.
- [28] C. Douillard, M. Jézéquel, C. Berrou, D. Electronique, A. Picart, P. Didier, and A. Glavieux, "Iterative correction of intersymbol interference: Turbo-equalization," *European Transactions on Telecommunications*, vol. 6, no. 5, pp. 507–511, 1995.
- [29] H. Ye and G. Y. Li, "Initial Results on Deep Learning for Joint Channel Equalization and Decoding," in *2017 IEEE 86th Vehicular Technology Conference (VTC-Fall)*, 2017, pp. 1–5.
- [30] W.-C. Tsai, C.-F. Teng, H.-M. Ou, and A.-Y. A. Wu, "Neural Network-Aided BCJR Algorithm for Joint Symbol Detection and Channel Decoding," in *2020 IEEE Workshop on Signal Processing Systems (SiPS)*, 2020, pp. 1–6.
- [31] W. Henkel, N. S. Islam, and M. A. Leghari, "Joint Equalization and LDPC Decoding," in *2019 11th International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*, 2019, pp. 1–5.
- [32] P. Robertson, E. Villebrun, and P. Hoeher, "A comparison of optimal and sub-optimal MAP decoding algorithms operating in the log domain," in *Proceedings IEEE International Conference on Communications ICC '95*, vol. 2, 1995, pp. 1009–1013 vol.2.
- [33] S. ten Brink, "Convergence behavior of iteratively decoded parallel concatenated codes," *IEEE Transactions on Communications*, vol. 49, no. 10, pp. 1727–1737, 2001.
- [34] F. Brannstrom, L. Rasmussen, and A. Grant, "Convergence analysis and optimal scheduling for multiple concatenated codes," *IEEE Transactions on Information Theory*, vol. 51, no. 9, pp. 3354–3364, 2005.
- [35] J. Hagenauer, "The turbo principle in mobile communications," *Proc. ISITA, XI'AN, Peoples Republic of China, Oct. 2002*, 2002.
- [36] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [37] A. Ashikhmin, G. Kramer, and S. ten Brink, "Extrinsic information transfer functions: model and erasure channel properties," *IEEE Transactions on Information Theory*, vol. 50, no. 11, pp. 2657–2673, 2004.
- [38] G. Böcherer, "Achievable rates for probabilistic shaping," *arXiv preprint arXiv:1707.01134*, 2017.
- [39] S. Cammerer, F. Ait Aoudia, S. Dörner, M. Stark, J. Hoydis, and S. ten Brink, "Trainable Communication Systems: Concepts and Prototype," *IEEE Transactions on Communications*, vol. 68, no. 9, pp. 5489–5503, 2020.
- [40] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *International Conference on Artificial Intelligence and Statistics*, 2010.
- [41] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [42] D. D. Falconer and F. R. Magee, "Adaptive channel memory truncation for maximum likelihood sequence estimation," *The Bell System Technical Journal*, vol. 52, no. 9, pp. 1541–1562, 1973.
- [43] F. Rusek and A. Prlja, "Optimal Channel Shortening for MIMO and ISI Channels," *IEEE Transactions on Wireless Communications*, vol. 11, no. 2, pp. 810–818, 2012.
- [44] G. Colavolpe and G. Germini, "On the application of factor graphs and the sum-product algorithm to isi channels," *IEEE Transactions on Communications*, vol. 53, no. 5, pp. 818–825, 2005.
- [45] I. Santos, J. J. Murillo-Fuentes, and P. M. Olmos, "Block expectation propagation equalization for ISI channels," in *2015 23rd European Signal Processing Conference (EUSIPCO)*. IEEE, 2015, pp. 379–383.
- [46] M. Lian, F. Carpi, C. Häger, and H. D. Pfister, "Learned Belief-Propagation Decoding with Simple Scaling and SNR Adaptation," in *2019 IEEE International Symposium on Information Theory (ISIT)*, 2019, pp. 161–165.
- [47] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [48] A. Roumy, I. Fijalkow, and D. Pirez, "Joint equalization and decoding: why choose the iterative solution?" in *Gateway to 21st Century Communications Village. VTC 1999-Fall. IEEE VTS 50th Vehicular Technology Conference (Cat. No.99CH36324)*, vol. 5, 1999, pp. 2989–2993 vol.5.
- [49] L. Szczecinski and A. Alvarado, *Bit-interleaved coded modulation: fundamentals, analysis and design*. John Wiley & Sons, 2015.
- [50] J. G. Forney, "Maximum-likelihood sequence estimation of digital sequences in the presence of intersymbol interference," *IEEE Trans. Inf. Theory*, vol. 18, pp. 363–378, 1972.
- [51] H. Yuan, H. Yu, S. Gui, and S. Ji, "Explainability in Graph Neural Networks: A Taxonomic Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5782–5799, 2023.
- [52] L. Schmid, J. Brenk, and L. Schmalen, "Local message passing on frustrated systems," in *Uncertainty in Artificial Intelligence*. PMLR, 2023, pp. 1837–1846.
- [53] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, "ViterbiNet: A Deep Learning Based Viterbi Algorithm for Symbol Detection," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3319–3331, 2020.
- [54] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An Open-Source Library for Next-Generation Physical Layer Research," *arXiv preprint*, Mar. 2022.
- [55] R. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- [56] D. Donoho, "High-Dimensional Data Analysis: The Curses and Blessings of Dimensionality," *AMS Math Challenges Lecture*, pp. 1–32, 01 2000.
- [57] G. Caire, G. Taricco, and E. Biglieri, "Bit-interleaved coded modulation," *IEEE Transactions on Information Theory*, vol. 44, no. 3, pp. 927–946, 1998.
- [58] C. Douillard, M. Jézéquel, C. Berrou, D. Electronique, A. Picart, P. Didier, and A. Glavieux, "Iterative correction of intersymbol interference: Turbo-equalization," *European Transactions on Telecommunications*, vol. 6, no. 5, pp. 507–511, 1995.
- [59] R. Wiesmayr, C. Dick, J. Hoydis, and C. Studer, "DUIDD: Deep-Unfolded Interleaved Detection and Decoding for MIMO Wireless Systems," in *2022 56th Asilomar Conference on Signals, Systems, and Computers*. IEEE, 2022, pp. 181–188.
- [60] T. Koike-Akino, Y. Wang, D. S. Millar, K. Kojima, and K. Parsons, "Neural Turbo Equalization: Deep Learning for Fiber-Optic Nonlinearity Compensation," *Journal of Lightwave Technology*, 2020.
- [61] J. Clausius, S. Dörner, S. Cammerer, and S. ten Brink, "Serial vs. Parallel Turbo-Autoencoders and Accelerated Training for Learned Channel Codes," in *2021 11th International Symposium on Topics in Coding (ISTC)*, 2021.
- [62] M. Tuchler, A. Singer, and R. Koetter, "Minimum mean squared error equalization using a priori information," *IEEE Transactions on Signal Processing*, vol. 50, no. 3, pp. 673–683, 2002.
- [63] I. Santos, J. J. Murillo-Fuentes, E. Arias-de Reyna, and P. M. Olmos, "Turbo EP-Based Equalization: A Filter-Type Implementation," *IEEE Transactions on Communications*, vol. 66, no. 9, pp. 4259–4270, 2018.
- [64] P. G. Martinsson, V. Rokhlin, and M. Tygert, "A fast algorithm for the inversion of general Toeplitz matrices," *Computers & Mathematics with Applications*, vol. 50, no. 5, pp. 741–752, Sept. 2005.
- [65] I. Santos, J. J. Murillo-Fuentes, R. Boloix-Tortosa, E. Arias-de Reyna, and P. M. Olmos, "Expectation Propagation as Turbo Equalizer in ISI Channels," *IEEE Transactions on Communications*, vol. 65, no. 1, pp. 360–370, 2017.
- [66] S. Dörner, J. Clausius, S. Cammerer, and S. ten Brink, "Learning Joint Detection, Equalization and Decoding for Short-Packet Communications," *IEEE Transactions on Communications*, vol. 71, no. 2, pp. 837–850, 2023.
- [67] C. Han, H. Zhao, Z. Chen, and F. Wang, "Sparse Neural Network for Detection and Decoding of Non-Binary Polar-Coded SCMA," *IEEE Transactions on Wireless Communications*, vol. 22, no. 7, pp. 4475–4488, 2023.
- [68] R. Wiesmayr, S. Cammerer, F. A. Aoudia, J. Hoydis, J. Zakrzewski, and A. Keller, "Design of a Standard-Compliant Real-Time Neural Receiver for 5G NR," *arXiv preprint arXiv:2409.02912*, 2024.
- [69] J. Ney, C. Füllner, V. Lauinger, L. Schmalen, S. Randel, and N. Wehn, "From Algorithm to Implementation: Enabling High-Throughput CNN-Based Equalization on FPGA for Optical Communications," in *Embedded Computer Systems: Architectures, Modeling, and Simulation*, C. Silvano, C. Pilato, and M. Reichenbach, Eds. Cham: Springer Nature Switzerland, 2023, pp. 158–173.
- [70] P. J. Freire et al., "Implementing neural network-based equalizers in a coherent optical transmission system using field-programmable gate arrays," *Journal of Lightwave Technology*, vol. 41, no. 12, pp. 3797–3815, 2023.